

OSCAR - Tester User Guide

How to point OSCAR at an OSDM provider and build, run, and read a conformance test.

Reflects collection **OTST_V2.0.12** / server **1.11.55** / release **2026.83**. Labels in **bold** match the onscreen controls. When the UI and this guide disagree, the UI wins - please open an issue so we can update the guide.

Table of contents

- 1. [Concepts & roles](#)
- 2. [Onetime setup](#)
- 3. [Provider setup - Framework, Test data & Discovery](#) - 3.1 Test Framework (capabilities) - 3.2 Train sets - 3.3 Journeys - 3.4 Timetable Discovery
- 4. [Authoring a scenario](#)
- 5. [Running tests](#)
- 6. [Reading the report](#)
- 7. [Reference tables](#)
- 8. [Troubleshooting](#)

1. Concepts & roles

What OSCAR does. OSCAR drives a provider's OSDM API through realistic sale / refund / exchange flows and checks each response against the OSDM spec. A run produces a **report**: a tree of pass/fail **assertions** plus the full **HTTP traffic** (every request and response).

The hierarchy. Everything hangs off four nested things - learn these names, the rest of the guide uses them:

Company Test Framework Scenario(s) Run Report
(the (what the (one concrete (one (assertions
provider + vendor CAN do - test: an O&D, execution) + HTTP
api_base) the "menu") a date, options) traffic)

- **Company** - the provider under test. Holds the **api_base** (the OSDM API root URL) and the percompany **concurrency limit**.
- **Test Framework** - declares the vendor's **capabilities** (ticket types, seatmap support, ancillaries, offer criteria...). It is the *menu*: it **gates** what a scenario is allowed to switch on.
- **Scenario** - one concrete test you author from that menu.
- **Run** - one execution of one or more scenarios.
- **Report** - the result.

Roles (set by an administrator; they decide what you can see and do):

Role	What it is	Sees
company_user (tester)	Daytoday tester for one company. Keeps their own OSDM credentials.	Their own runs in their company.
test_manager	Company lead.	All of the company's runs/reports; can share a report with the certifier.
certification_user (certifier)	UICside reviewer, no company of their own.	Only the reports a test_manager has explicitly shared .
administrator	Platform operations & security.	Manages users/companies/config. Does not read test data (privacystrict, issue #60).

If you are a tester, you live mostly in sections **3-6** below.

2. Onetime setup

Do these once before your first run.

1. **Sign in** to the OSCAR web app with the account your administrator created.
2. **Set your OSDM credentials** (Profile / API config). Credentials are **per user**: each tester authenticates to the company's OSDM API with their own client id / secret (or token). OSCAR requests the access token for you at run time when the sandbox needs one - you don't paste a token by hand.
3. **Confirm the company api_base** points at the right sandbox (e.g. <https://osdm-5.platform.bileto.zone/api>). This is company level config; a **test_manager/admin** sets it.
4. **Pick the environment / sandbox** for the run (e.g. Bileto, Sqills, Benerail, ...). The collection autoselects the matching accesstoken request for that sandbox.

Credentials never leave the server in the clear. OSDM credentials and the framework/data are encrypted at rest; auth request/response bodies are redacted in every report.

3. Provider setup - Framework, Test data & Discovery

Everything in this section is the **provider setup** a **test_manager** does **once** (and revises as they learn the vendor): the **capabilities** the vendor supports, and the **trains/journeys** to test against. Scenarios (4) are then authored *on top of* this. Plain testers (**company_user**) consume this setup - the train/journey/discovery edit controls are hidden from them.

3.1 Test Framework (capabilities)

The framework tells OSCAR **what the vendor can do**. It does **not** run anything - it defines the menu that scenario authoring is limited to.

Capability	Field	What it means / gates
Ticket types	<code>rail.ticketTypes</code>	IRT, NRT_OPTIONAL_RESERVATION , ... Declaring a reservationbearing type is what makes place selection available to scenarios.
Graphical seat map?	Seat Selection -> "Does the system offer a graphical seat map?" <code>(placeSelection.seatMap)</code>	Tick if travellers can pick a specific seat. Off => scenarios cannot pick seats at all.
Supported seatselection modes	the pills revealed under the box <code>(placeSelection.supportedModes)</code>	Tick ** Seat map at offer and/or ** Add reservation to a booking - declare the one(s) this vendor actually supports. A scenario's mode picker (4.6) offers exactly these and nothing else.
Offer criteria	<code>offerCriteria.{serviceClasses, travelClasses, requestedOfferParts, flexibilities, offerMode, currency}</code>	The allowed values a scenario's offer search may use.
Fulfillment	<code>fulfillment.{media, types}</code>	Allowed fulfilment media/types (e.g. PDF_A4 / ETICKET).
Ancillaries	<code>ancillaries[]</code>	The catalog of ancillary types the vendor sells. Declaring >=1 enables the Add ancillary action. Defined once, reusable per train (issue #130).
Passenger types & ages	<code>passengerTypes, passengerAgeRanges</code>	e.g. ADULT 26-99, CHILD 4-15 .
Concurrency limit	percompany	Max parallel runs OSCAR fires at this vendor. Lower it if the vendor is fragile under load (see 8).

The golden rule: a scenario can only switch on what the framework authorises. If a control is greyed out during scenario authoring, the reason is almost always "enable it in the Test Framework first."

3.2 Test data - train sets (routes + timetables)

A scenario doesn't carry its own train - it **references** one of the **train sets** you define here (the **** Train Resources**** section). A train set is:

- a **route** - **Label** (short display name, e.g. *"Squills IC BAS/AMS"*), **Operator Code** (`urn:uic:rics:NNNN`), **Origin / Destination station URN**, and optional **Product category** (ref / name / short name);
- a **Services (timetable)** - one or more individual departures that run that route, each with a **vehicle number** and **departure / arrival** times;
- **Operating days** - setlevel (applies to every service in the set); **empty = daily**. Lets you model "runs Mon-Fri only", etc.

Controls:

Button	Does
** Add Train**	Create a new train set and fill the route + services by hand.
** Duplicate**	Copy an existing set, then tweak the vehicle # / times - the fast way to add a sibling departure or a nearidentical route.
** Save all trains**	Persist every set you've opened/edited in one go.

A scenario then points at a set via its **Trip** (requirement) - **tripType SEARCH** (let the vendor find the train from O&D + date) or **SPECIFICATION** (pin the exact train/legs).

3.3 Journeys (multileg)

The **** Journeys section chains train sets into a reusable multileg itinerary (e.g. *Basel -> Amsterdam -> Paris*). Build it once with **** Add Journey** (or **** Duplicate**), and a scenario can then Apply a Journey to fill all** its legs at once instead of typing each leg. Ideal for testing multileg / connection offers consistently.**

3.4 Timetable Discovery

Don't know which trains a sandbox actually runs? Use **** Discover timetable**** (in the Train Resources section) to reverseengineer them:

1. Enter an **Origin** and **Destination** (UIC URN or bare code - both work).
2. Set **Days to scan** (1-14, default 7).
3. Click **Discover**.

OSCAR queries the sandbox serverside (**POST /offers**, with a **/trips-collection** fallback) across the next N days, harvests every train it offers, and **creates/updates the train sets** accordingly - including the **travel/service classes and ancillaries** the vendor actually returns, and splitting sets by **operatingdays pattern** (so a weekdayonly train and a daily train become separate sets). **Your manual edits are preserved** - discovery merges, it never clobbers. It's the fastest way to seed a brandnew provider's test data.

Discovery and train/journey editing are **test_manager** actions; they're hidden for plain testers.

4. Authoring a scenario

A scenario is built in the **Scenarios** section. The good news (issue #172): **only three things are required** - everything else is optional and defaults sensibly.

4.1 The minimal scenario

Required	Notes
Origin	A station reference. UIC URNs (<code>urn:uic:stn:...</code>) and vendor URNs (<code>urn:x_<vendor>:stn:...</code>) are both accepted.
Destination	As above.
Departure date	Resolved dynamically at run time (<code>%TRIP_DATE%</code>), so a saved scenario never goes stale.

That alone is a valid **SALE** search. Add the options below only when you want to exercise them.

4.2 Scenario type

Type	Flow exercised
SALE	Offer -> booking -> (optional steps) -> fulfillment. The default.
REFUND	Sale, then refundoffer -> refund. Needs an overrideCode (PAYMENT_FAILURE / DISRUPTION).
EXCHANGE	Sale, then exchangeoffer -> exchange operation.

4.3 Offer criteria (optional)

Narrow what you ask the vendor for. Each value is constrained to what the framework authorised.

- **Currency** (e.g. **EUR**) - *recommended even though optional*: some strict vendors **400/500** on an empty **offerSearchCriteria** with no currency.
- **Service class** - **STANDARD**, **BEST**, **HIGH**, **BASIC**, **ANY_CLASS**.
- **Travel class** - **FIRST**, **SECOND**, **ANY_CLASS**.
- **Requested offer parts** - **ADMISSION**, **RESERVATION**, **ANCILLARY**, **FARE_***, **CONTINUOUS_SERVICE**, **ALL**. (Include **RESERVATION** if you intend to pick seats.)
- **Flexibility** - **FULL_FLEXIBLE**, **SEMI_FLEXIBLE**, **NON_FLEXIBLE**.
- **Offer mode** - **INDIVIDUAL**, **COLLECTIVE**.

4.4 Return trip (optional)

Leave empty for a **oneway**. To make it a **return**, set:

- **Return offset (days)** - **0** = same day, **1**, **2**, ... Default suggestion is **2** (covers night trains). The return date is derived as *outbound departure date + offset*, so it tracks the dynamic departure date.
- **Return time** (optional **HH:MM**) - overrides the timeofday; otherwise the outbound departure time is mirrored.

Under the hood this becomes **returnSearchParameters.inwardReturnDate** and triggers the **twostep return** (outbound offer -> inward offer -> roundtrip booking). If a vendor rejects a combined twooffer booking, OSCAR automatically falls back to two separate bookings and records a trackable finding (issue #180).

4.5 Salesflow actions (optional optin steps)

These are extra steps inserted into a **SALE** flow. **All default OFF** - opt in to each one you want to test. An action that the framework doesn't authorise is shown disabled with the reason.

Action	What it does	Requires
PATCH passengers	Updates passenger details between booking and fulfillment.	-
Place selection	Picks specific seats (see 4.6).	A reservation ticket type + the graphical seat map ticked in the framework (3).
Add ancillary	Adds an ancillary offer part to the booking.	>=1 ancillary in the framework.
GET booking	Reads the booking back for a consistency check.	-
Delete ancillary	Removes a previously added ancillary (reverse path).	>=1 ancillary.

4.6 Seatselection mode (when Place selection is on)

This is where the **Test Framework** and the **scenario** connect. It's a twoplace setup - declare the capability once on the framework, then pick from it on each scenario:

In the Test Framework -> Seat Selection (3): 1. Tick **"Does the system offer a graphical seat map?"** (`placeSelection.seatMap`). 2. Under **Supported seatselection modes**, tick the option(s) the vendor really supports - **** Seat map at offer and/or ** Add reservation to a booking** (one or both).

In the scenario (here): 3. Turn on the **Place selection** salesflow action (4.5). 4. Pick **one** mode from the **Seatselection mode** picker - it shows **exactly** the option(s) you ticked in step 2, nothing else.

If the framework lists only one mode, the scenario preselects it - **still click the pill so the choice is saved**. An unsaved/stale value can silently run the other flow (this is the cause of "I chose *Seat map at offer* but the offertime step never ran"; see 8).

The two modes - the same labels appear in the framework and the scenario:

Mode (<code>placeSelectionMode</code>)	Label (UI)	When the seat is chosen
<code>SEATMAP_AT_OFFER</code>	** Seat map at offer**	<i>Before</i> the booking. OSCAR GETs the offertime seat map, picks an available seat per passenger, and carries it into the <code>POST /bookings</code> . The seat may affect the price.
<code>ADD_TO_BOOKING</code>	** Add reservation to a booking**	<i>After</i> the booking. A reservation offer part is added to the existing booking (e.g. SNCF firstclass TGV).

Adaptive behaviour (issues #182/#184/#186/#188): - `SEATMAP_AT_OFFER` is the discoveryfriendly choice: it tries the offertime map and, **if that fails, automatically falls back** to the afterbooking path. So for an unknown vendor, authorise *both* modes in the framework and choose `SEATMAP_AT_OFFER`. - The seat picked is **availabilityaware** (occupied/reserved places are skipped) and **one seat per passenger**. There is intentionally **no "seatpassengerstogether"** optimisation yet. - A reservation must exist in the offer for a seat map to apply. If it doesn't, the seatmap step is skipped cleanly ("not applicable") rather than erroring.

4.7 Passengers & fulfillment (optional)

- Passengers** - type (`PERSON`, `BICYCLE`, `DOG`, `PRM`, ...), date of birth, and optional patch values (name/email/phone) used by the `PATCHpassengers` step.
- Fulfillment** - media (`PDF_A4`, `UIC_PDF`, ...) and type (`ETICKET`, ...), constrained to the framework's `fulfillment`.

5. Running tests

- Run one scenario** or **Run the collection** (all scenarios in `scenariosToRun`, from the top). A run starts at `GET /versions`, which also resets all perrun state.
- Concurrency** - OSCAR can run several sessions in parallel up to the global limit **and** the percompany limit (the lower wins). If a vendor misbehaves under parallel load, set that company's limit to **1** to serialise.
- You'll get a **run** in the dashboard with a live status; when it finishes the **report** and a downloadable **JSON results** artifact are attached.

6. Reading the report

The run detail page has two collapsible cards (everything starts collapsed):

Assertions

Two levels: **main area (suite)** -> **endpoint (request)** -> individual assertions, with pass/fail counts at each level. Filters let you show all / failed only.

- A green count = conformant. A red **N failed** on an otherwise **200** request means **OSDM compliance assertions** failed - i.e. the *response* deviates from the spec. That's a **finding about the vendor**, not an OSCAR error.
- Watch for the deliberately **trackable** assertions:
- `[OSDM] Vendor serves a pre-booking (OFFER-context) seat map`

- [OSDM] Vendor serves a post-booking (BOOKING-context) seat map
- [OSDM] Vendor supports booking multiple offers (round trip) in one booking

These are written to **fail loudly** when a vendor lacks an optional capability, so you can filter the report for "what this vendor can't do." A matching [VENDOR GAP] line appears in the execution log. The flow still completes via the adaptive fallback.

HTTP Traffic - Request & Response

Same suite -> endpoint structure. Click any endpoint to lazily load its full request body + response body (prettyprinted) and headers. Use this to confirm exactly what OSCAR sent (e.g. that `resourceId` resolved, or that `placeSelections.places[]` has the right shape).

HTTP status vs. assertion failures are different things. A request can be 200 (HTTP OK) and still have failed assertions (spec deviations), and viceversa.

7. Reference tables

7.1 Scenario parameters -> where they go

Parameter	Values	Affects request
Scenario type	SALE / REFUND / EXCHANGE	Which flow runs
Origin / Destination	station URN	POST /offers trip
Departure date	dynamic (%TRIP_DATE%)	POST /offers departureTime
Return offset (days) + return time	0, 1, 2 ... + HH:MM	returnSearchParameters.inwardReturnDate
Currency	EUR, ...	offerSearchCriteria.currency
Service class	STANDARD BEST HIGH BASIC ANY_CLASS	offerSearchCriteria
Travel class	FIRST SECOND ANY_CLASS	offerSearchCriteria
Requested offer parts	ADMISSION RESERVATION ANCILLARY FARE_* CONTINUOUS_SERVICE ALL	offerSearchCriteria
Flexibility	FULL_FLEXIBLE SEMI_FLEXIBLE NON_FLEXIBLE	offerSearchCriteria
Offer mode	INDIVIDUAL COLLECTIVE	offerSearchCriteria
Override code	PAYMENT_FAILURE DISRUPTION	refund/exchange request
Salesflow actions	patchPassengers, placeSelection, addAncillary, getBooking, deleteAncillary	inserts the matching step
Seatselection mode	SEATMAP_AT_OFFER / ADD_TO_BOOKING	seatmap step + placeSelections
Passenger type	PERSON BICYCLE DOG PRM ...	passenger specs
Fulfillment type / media	ETICKET... / PDF_A4...	requestedFulfillmentOptions

7.2 Saleflow step -> OSDM request

Step (collection)	OSDM call
01 POST Get Offer	POST /offers
01b POST Get Return Offer	POST /offers (inward leg)
08 GET Place Maps	GET /availabilities/place-map?contextType=OFFER

08b GET Place Map Post-Booking	GET /availabilities/place-map?contextType=BOOKING
02 POST Create Booking	POST /bookings
09 POST Add Reservation	POST /bookings/{id}/booked-offers/{id}/{offer-parts\ reservations}
03 PATCH Multi Passenger	PATCH .../passengers
06 POST Fulfillments	POST /fulfillments

8. Troubleshooting

Symptom	Likely cause	Fix
400 "Invalid request content" on POST /offers	offerSearchCriteria: {} empty (no currency) on a strict vendor	Add a currency to the scenario's offer criteria.
400 with resourceId=%7B%7BreservationId%7D%7D in the placemap URL	The offer had no reservation part , so reservationId was empty	Use a train/class that includes a reservation; or expect the seat map to be skipped as "not applicable".
400 "Invalid request content" on POST /bookings after a seat pick	Malformed placeSelections.places[]	Should be fixed (OSDM SelectedPlace = {coachNumber, placeNumber, passengerRef}). If it recurs, capture the booking body and report it.
Intermittent 500, same request sometimes works	Vendor fragile under parallel load	Lower the company concurrency limit to 1.
501 parameter_not_supported	Vendor conformantly declines an optional capability (e.g. no BOOKINGcontext map)	Expected; the adaptive fallback continues. It's a capability finding, not a bug.
Seat map step (08) missing from the report on a SEATMAP_AT_OFFER scenario	The scenario's mode wasn't actually saved (stale value), so it ran ADD_TO_BOOKING	Open the scenario, click the Seat map at offer pill, save, rerun.
Many "failed" assertions on 200 responses	OSDM compliance deviations in the vendor's responses	These are the conformance findings - review them; they're the point of the tool.

Glossary

- **OSDM** - Open Sales and Distribution Model (the rail retailing API standard).
- **Offer / Booking / Fulfillment** - the three OSDM sale stages.
- **Admission / Reservation / Ancillary** - kinds of *offer part* (the ticket, the seat, the extras).
- **Trackable assertion** - a deliberatelyfailing check used to flag a vendor capability gap so it's easy to filter in the report.
- **Adaptive fallback** - OSCAR automatically takes an alternate path when a vendor rejects the specpreferred one (e.g. seat after booking; two bookings for a round trip).